Personal Portfolio Website Design & Implementation Specification

User: Portfolio Owner

May 18, 2026

1 Document Purpose

This document specifies the design and implementation guidelines for a personal portfolio website consisting of one home page and five content pages: Academic, Professional, Projects, Readings (Books), and Photography.

The document is intended to be:

- **Executable as a spec for an AI assistant:** The AI should be able to infer page structure, data schemas, and UI behaviors cleanly from this text.
- **Actionable for developers:** Clear enough that a front-end or back-end developer can implement the site without additional high-level clarification.

The specification assumes a modern web stack featuring:

- A React-based frontend (or Next.js) with rich animations, micro-interactions, and mouse-hover physics.
- Either a Django-based backend API providing JSON payloads, or a content-first static setup where long-form text content lives in Markdown format with front matter.
- Source code hosted securely on GitHub with connected continuous deployment workflows.

2 High-Level Architecture

2.1 Core Goals

- Create a visually appealing, industry-competitive portfolio website with interactive UI components.
- Make content updates (text records, book ratings, photo metadata) easy and low-friction, eliminating the need for frequent code changes.
- Provide highly structured data models for Academic, Professional, Projects, Books, and Photography pages.
- Maintain a strict separation of content (text data, metadata arrays) from application behavior and presentation logic (React hooks, components, styling systems).

2.2 System Overview

- **Frontend:** React application responsible for client-side routing, view rendering, and transitions. Uses a utility-first styling system (e.g., Tailwind CSS) for consistent responsive design, and an animation engine (e.g., Framer Motion) to power custom interactive elements, page swaps, and modal overlays.
- **Backend / Content Layer:**

  - *Option A:* Django and Django REST Framework (DRF) supplying dynamic JSON APIs for all structured degrees, jobs, projects, books, and photo entries.
  - *Option B:* Content-first architecture utilizing localized Markdown files compiled at build time into static JSON objects consumable by the React tree.
- **Deployment:** Version control managed via GitHub. The frontend layer is deployed to a static hosting environment (e.g., Vercel, Netlify, or GitHub Pages), while any required back-end application layers are contained in a PaaS or container engine.

2.3 Routing and Pages

The application exposes six core routes mapping directly to the main system architecture:

- `/`: Home (Split-pane dashboard)
- `/academic`: Academic
- `/professional`: Professional (Profiles)
- `/projects`: Projects
- `/books`: Books / Readings
- `/photography`: Photography (Photo)

The global navigation header displays the corresponding entry links: **Academic**, **Profiles**, **Projects**, **Books**, and **Photo**.

3 Content Model

This section defines the logical data schemas across modules, implementable as Django models, TypeScript interfaces, or structured front matter fields inside Markdown files.

3.1 Shared Concepts

3.1.1 Tech Stack Item

- `id`: Unique identifier string or integer sequence.
- `name`: Technology name text field (e.g., "Python", "R", "Tailwind").
- `logo_url`: Resource URL or asset key mapping to the technology logo.
- `category`: Optional grouping field (`language`, `framework`, `tool`).
- `level`: Optional proficiency weight or rendering importance score.

Usage: Linked uniformly inside Academic (course tags), Professional (job records), and Projects (system stacks).

3.2 Academic Module

3.2.1 Degree

- `id`: Unique database identifier key.
- `institution`: Educational institution title text (e.g., "CU Denver", "MSU Denver").
- `degree_type`: Academic credential category string (e.g., "Masters", "Bachelors").
- `majors`: Comma-separated or structured list layout tracking specialized paths (supports configurations such as two majors and one minor).
- `concentration`: Optional discipline specialization string.
- `start_date`, `end_date`: Date tracking fields.
- `overview_md`: Markdown summary detailing specific achievements and pathways.

3.2.2 Course

- `id`: Unique course row identifier.
- `degree_id`: Foreign key reference mapping directly to a specific `Degree` instance.
- `name`: Title of the course.
- `code`: Full academic catalog code (e.g., "CS 3010").
- `short_description_md`: Core description block explaining key objectives.
- `tags`: Keyword lists indicating foundational disciplines (e.g., `statistics`, `ML`, `programming`).

3.3 Professional Module

3.3.1 Job

- `id`: Unique job position record identifier.
- `title`: Specific employment role title text.
- `organization`: Employing company or institution title.
- `location`: Geographic location text field.
- `start_date`, `end_date`: Range fields for employment duration.
- `description_md`: Detailed multi-line Markdown tracking engineering contributions and deliverables.
- `tech_stack`: Array reference linking to relevant `Tech Stack Item` nodes.

3.4 Projects Module

3.4.1 Project

- `id`: Unique project repository code identifier.
- `title`: Name of the software system or engineering research endeavor.
- `short_description_md`: Abstract or brief overview capsule (1–3 lines).
- `long_description_md`: Deep-dive text description rendered inside modal cards or separate views.
- `tech_stack`: Array of related `Tech Stack Item` entries.
- `links`: Optional dictionary mapping code repositories, live production URLs, or published academic papers.
- `status`: Structural track configuration tracking state (e.g., "in progress", "completed").

3.5 Books / Readings Module

3.5.1 Book

- `id`: Unique book review identifier.
- `title`: Core title text of the literature.
- `author`: Full name text of the author.
- `category`: Discrete choice string selecting exactly from {`Learning, Nonfiction, Fiction`}.
- `rating`: High-precision numeric evaluation field bounded strictly to the scale `[0.0, 10.0]`.
- `cover_image_url`: URL endpoint string pointing to book graphics or structural placeholder.
- `started_at`, `finished_at`: Chronological reading dates.
- `summary_md`: Main content summary written by the site owner.
- `notes_md`: Secondary notes section for extended commentary or reference points.
- `tags`: Structural reference list of focus themes (e.g., `statistics`, `design`).

3.6 Photography Module

3.6.1 Device

- `id`: Hardware platform identifier.
- `name`: Captured reference label (e.g., "Device 1", "Device 2", "Device 3").
- `type`: Category label defining hardware characteristics (e.g., `camera`, `phone`).

3.6.2 Photo

- `id`: Unique image asset catalog identifier.
- `image_url`: Image delivery asset hosting link.
- `location`: Geographic location tag representing capture context.
- `year`: Calendar year integer capturing creation timeline.
- `device_id`: Reference pointer mapping to specific system `Device` metadata.
- `description_md`: Inline narrative context field.
- `tags`: Text array indexing artistic themes or technical parameters.

4 Frontend Design and Interaction Specification

4.1 Global Layout

- **Navigation Bar:** Affixed persistently to the top edge on desktop layouts; collapses into an interactive touch menu overlay on mobile screen configurations. Houses explicit routing targets linking to Home, Academic, Profiles (Professional), Projects, Books, and Photo.
- **Footer:** Sticky lower section containing structured external contact links pointing to GitHub, LinkedIn, and LeetCode, alongside an optional mailto endpoint.
- **Typography and Palette:** Employs 2–3 coordinating font families mapping clean headers, running body passages, and optional monospaced logs. Incorporates an integrated accent theme color to shade badge containers, button hover states, and interface links.

4.2 Home Page Split-Pane Framework

The home page breaks away from conventional linear rows, executing an asymmetrical, two-column split screen matching structural drawing coordinates:

4.2.1 Left Pane: Fixed Profile Sidebar

- **Circular Avatar:** A prominent circular image container centering a professional portrait at the top of the sidebar.
- **Name Header:** High-contrast textual header layout presenting the owner's name prominently.
- **Short Bio Frame:** A concise text block describing current technical roles, research focus areas, and primary engineering competencies.
- **External Links Cluster:** A dedicated vertical list container cleanly exposing secondary anchors:
  - `- Github`: Secure link directly to coding repositories.
  - `- linked in`: Links to professional network profile.
  - `- Leetcode`: Links to active algorithmic training metrics.

4.2.2 Right Pane: Main Navigation Guide

- **Header Banner:** Displays the welcome banner message: "Website Navigation Guide".

- **Section Showcase Cards:** Contains individual card tiles summarizing what each main route offers (Academic, Professional, Projects, Books, Photography).
- **Interaction and Navigation:** Cards react to mouse-hover inputs via gentle structural elevations. Clicking any tile triggers clean client-side routing to the selected page.
- **Footer Footnote Anchor:** A distinct lower partition separating a specialized "Read more" label mapping out underlying "API documentation; tech stack" references.

4.3 Academic Page Layout

A clean vertical timeline layout displaying institutional credentials, structured as a multi-column dashboard stack.

4.3.1 Institutional Degree Layout

- **Graduate Block (Masters @CuDenver):** Highlights degree and specialization paths (e.g., M.S. in Applied Mathematics). Renders an abstract overview via Markdown text alongside a structured coursework grid.
- **Undergraduate Block (Bachelors @MSU Denver):** Clearly tracks multi-disciplinary tracks (e.g., 2 majors and 1 minor configuration). Hosts an independent tabular matrix index of fundamental coursework.

4.3.2 Coursework Grid Interactions

- Course blocks are compiled inside grid cells featuring full item code and title metadata.
- User mouse cursor hover states over a course cell trigger an animated tooltip overlay displaying a brief overview description or localized tags.
- Incorporates optional filtering chip pills along header frames to parse visibility by focus areas (e.g., `statistics`, `computational biology`).

4.4 Professional Page Layout

An interactive resume block topped by an analytical background synthesis block.

4.4.1 Content Layout

- **Section Intro:** Main page text wrapper labeled "Overview - Professional Background" (sketched as *"Backynd"*).
- **Job Card Elements:** Sequential vertical list arrangement grouping jobs chronologically (Job 1, Job 0). Each element maps:
  - Structural header tracking Position title and Organization @ Location settings.
  - Chronological duration range fields.
  - Deep description block displaying rich Markdown bullets detailing core engineering tasks and team impacts.
  - Dedicated lower row containing technological tracking tools enclosed in animated visual badges.

4.4.2 Evasive Tech Stack Bubbles Behavior

Each skill badge operates within an isolated animation boundary to produce custom physics micro-interactions:

- **Idle Floating State:** Bubbles oscillate vertically using looping sinusoidal curves paired with slight horizontal noise to generate a natural buoyancy look.

- **Evasive Cursor Interaction:** When a user's cursor crosses proximity boundaries near a bubble, the frontend layout engine applies physics vectors, dynamically shifting the badge element away from the cursor.
- **Content Separation Rules:** Adding a technology element onto a job profile only requires adjusting data JSON files; animation code blocks remain untouched.

4.5 Projects Page Layout

A showcase grid layout engineered for deep technical reviews.

4.5.1 Project Matrix Architecture

- **Overview Banner:** Top textual frame mapping development strategies and live version control metrics.
- **Project Cards:** Grid blocks cataloging standalone platforms (Project 1, Project 0). Each block maps a project title, a short 1–3 line summary description abstract, and a dedicated tech stack bubble row reusing the looping, cursor-evading physics.

4.5.2 Interactions

- Pointer hover frames elevate card shadows and apply a subtle color transition accent along structural borders.
- Click inputs smoothly blur out background contexts and open full-screen central modal dialog frames containing detailed design notes, architecture graphs, and direct deployment hyperlinks.

4.6 Books Page Layout

A clean digital book shell layout organizing reading materials via metadata tags and review scores.

4.6.1 Control Dashboard and Toolbar

- **Description Header:** Text block introducing annual reading targets, core study focuses, and active reading scopes.
- **Category Filter Chips:** Interactive selection tabs grouping lists cleanly into structural buckets: `[Learning]`, `[Nonfiction]`, and `[Fiction]`.
- **Sorting Dropdown:** Integrated selection element labeled `[Sorting]` that reorders list items by precise rating scales, completion timelines, or alphabetical orders.

4.6.2 Cards and Overlay States

- **Reading Shelf Grid:** Arranges book records into a responsive thumbnail layout presenting cover art imagery, main title text, and author data.
- **Hover Score Badge:** Moving a pointer over a book container surfaces a styled overlay pill containing a precise evaluation score ranging from `0.00` to `10.00`.
- **Summary Overlay Modals:** Clicking an active record pauses the lower document frame behind a dark blur, opening a comprehensive card displaying complete date scopes, custom markdown review entries (`summary_md`), and detailed study lists (`notes_md`).

4.7 Photography Page Layout

An asset gallery built around precise metadata indexing.

4.7.1 Media Navigation Interface

- **Theme Intro:** Descriptive block highlighting focal settings, artistic framing styles, and active equipment.
- **Filter/Sort Toolbar System:** A unified toolbar interface designed to parse the photo grid through custom parameters:
  - `Devices`: Filters image records to isolate pictures matching individual configurations (`1`, `2`, or `3`).
  - `Location`: Dropdown checklist grouping assets by capture coordinates.
  - `Year`: Groups assets chronologically.
  - `Sorting`: Toggle switch to reorder thumbnail layouts dynamically.

4.7.2 Presentation Layout

- **Thumbnail Grid Matrix:** A responsive image container compiling asset previews cleanly across varying viewports.
- **Lightbox Viewport:** Selecting an asset triggers a lightbox component displaying the high-resolution photo accompanied by its description and artistic tags.

5 Content Management Strategy

5.1 Markdown for Large Text

All large-form text or frequently edited prose records are isolated from the layout and managed strictly within standard Markdown fields:

- Section entries including Degree overviews and Academic milestones.
- Comprehensive professional Job records and contribution bullet arrays.
- Project design blueprints, architectural narratives, and system summaries.
- Book critique logs, reference notes, and creative descriptions.

The frontend framework handles text conversions at runtime through an optimized Markdown rendering module.

5.2 Editing Workflow

- **Django CMS Setup:** If deployed via Option A, content edits are handled via the native Django administration backend using rich text markdown input frames. The client frontend updates instantly by querying endpoints over the REST API layer.
- **Static Content Setup:** If deployed via Option B, edits are made by modifying localized `.md` files in a standard text editor. Changes are committed and pushed directly to the GitHub repository, which triggers automated CI/CD build actions to compile and serve the latest version.

6 Backend API (If Django is Used)

6.1 API Design Principles

- Deliver structural payloads using standard REST architectural design and clean JSON formats.
- Provide read-only list views and single-item detailed data endpoints for all system schemas:
  - `/api/degrees/` — Lists academic history logs and nested coursework entries.
  - `/api/jobs/` — Dispatches professional chronology records and related tech tags.

  - `/api/projects/` — Exposes development project logs and resource links.
  - `/api/books/` — Delivers book review elements with support for category parsing filters.
  - `/api/photos/` — Dispatches asset records, photo paths, and filter fields.
- Support standard URL query parameters on reading and photo paths to handle filtering and sorting calculations server-side.

6.2 Example Responses

- **Book JSON Node Structure:** Matches the complete key structure containing: `title`, `author`, `category`, `rating`, `summary_md`, `notes_md`, and `tags[]`.
- **Job JSON Node Structure:** Matches the full structural contract containing: `title`, `organization`, `location`, `description_md`, and `tech_stack[]`.

7 Frontend Technical Stack

7.1 React Application Layer

- Core rendering architecture constructed using React with React Router managing clean client-side routing structures.
- State tracking handled locally via internal hooks, or delegated to a lightweight global store to synchronize responsive drawer flags or active category tabs.
- Layout styling implemented with Tailwind CSS utilities to ensure fluid, cross-device execution and standard design values.

7.2 Animations and Hover Effects

- Leverages Framer Motion primitives to orchestrate rich micro-interactions across components.
- Configures custom animation equations to drive floating tech stack bubbles using continuous sinusoidal positioning code.
- Maps custom cursor pointer distance computations inside event loops to adjust real-time evasive translation vectors on interactive nodes.
- Smoothly animates card transformations, modal scaling, list reordering, and background dim/blur transitions.

7.3 Accessibility Considerations

- Verifies that all hover-only data layouts (such as reading evaluation metrics) remain accessible via standard tab indexing and keyboard focus listeners.
- Supplies explicit ARIA landmark labels (`aria-modal`, `aria-expanded`) across custom modal dialogs, tab groups, and hamburger expansion triggers.
- Enforces strict color contrast criteria across backgrounds, copy texts, and badge assets to maintain legibility.

8 Deployment and GitHub Integration

8.1 Repository Structure

The system repository is organized either as a monorepo or split across micro-repositories according to content layer decisions:

- `/frontend` — Houses the source files for the React/Next.js application.
- `/backend` — Houses the backend Django framework configuration code (Option A).

- `/content` — Localized folder holding structured Markdown text records and static data files (Option B). End item configurations are committed securely to a master branch on GitHub.

8.2 Continuous Deployment

- Connects version control directly to automated compilation and delivery infrastructure (CI/CD pipeline actions).
- Merging or pushing codebase revisions into production branches immediately starts automation tasks that validate formatting, build frontend assets, compress media nodes, and distribute static assets out to hosting edge servers.

9 AI Agent Usage Guidelines

9.1 AI Responsibilities

Artificial Intelligence tools leveraging this specification sheet should be capable of:

- Generating new data node entries (such as fresh project rows or book log components) that adhere precisely to the schemas defined in Section 3.
- Proposing layout modifications that follow the split-pane rules, card animations, and mouse avoidance behaviors.
- Writing and structuring comprehensive Markdown content summaries and experience bullets.
- Generating mock dataset objects to expedite localized prototyping.

9.2 Constraints

- The AI must not alter foundational architecture decisions (such as the split-pane setup or data-presentation separation) unless given explicit, overriding directions.
- Suggested code edits must respect data integrity rules, keeping behavior code separated from text data payloads at all times.